

USB Emulated Mass Storage Experiment

USB Emulated Mass Storage Experiment

- 1. USB Emulated Mass Storage Overview
- 2. Core Concepts
 - 2.1 USB MSC Protocol Stack
 - 2.2 WL Handle Handover Issue
 - 2.3 Wear-Leveling
 - 2.4 NOR Flash Writes Must Erase First
 - 2.5 Reason for ESP-IDF API Mixing
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code
 - 5.1 Write Preset File
 - 5.2 MSC Read/Write Callbacks
 - 5.3 `setup()` — WL Mount and MSC Startup
- 6. Operation Flow
 - 6.1 Compile and Flash
 - 6.2 Normal Phenomena
 - 6.3 Verification Methods
- 7. Common Issues

1. USB Emulated Mass Storage Overview

USB MSC (Mass Storage Class) makes ESP32-S3 emulate a USB flash drive, allowing PC to read and write without installing any driver. This experiment exposes a 1MB FAT partition on SPI Flash to PC via TinyUSB: creates `welcome.txt` in the partition after power-on, then starts the USB MSC device stack, and PC recognizes it as a readable and writable FAT-formatted flash drive.

Typical Application Scenarios:

Application Type	Examples
Data Export	Export sensor records as CSV files, read by unplugging flash drive style
Configuration Update	Modify config files on PC, insert device back for auto-activation
Firmware Upgrade	Drag firmware file into flash drive, device auto OTA after restart
File Exchange	PC driver-free file access for voice recorders, data loggers

2. Core Concepts

2.1 USB MSC Protocol Stack

USB MSC is one of the standard device classes defined by the USB organization, allowing embedded devices to emulate storage devices (flash drives, mobile hard drives). PC communicates with the device via standard SCSI command set without installing any driver.



SCSI Command to Project Implementation Mapping:

SCSI Command	Function	Project Implementation
READ(10) / READ(12)	PC reads sector data	<code>onRead(lba, offset, buf, size) → wl_read()</code>
WRITE(10) / WRITE(12)	PC writes sector data	<code>onwrite(lba, offset, buf, size) → wl_erase_range() + wl_write()</code>
START STOP UNIT	PC ejects/mounts media	<code>onStartStop(power, start, eject) → returns true</code>
INQUIRY	Query device info (vendor/product ID)	Auto-handled by TinyUSB, no callback needed
READ CAPACITY	Query disk capacity (sectors × 512)	Auto-handled by TinyUSB, no callback needed

The entire chain: PC File Manager → FATFS translates operations to sector read/write → SCSI commands → USB packets → TinyUSB → Callback functions → WL read/write → SPI Flash hardware.

2.2 WL Handle Handover Issue

The core design challenge of this experiment: **FFat and USB MSC cannot simultaneously hold the WL handle.**

Phase 1: write preset file

FFat.begin("storage") → Internally calls wl_mount() to get WL handle

File.write(welcome.txt) → writes to Flash via WL handle

FFat.end() → Internally calls wl_unmount() to release WL handle

Phase 2: USB MSC running

wl_mount(part, &wl_handle) → Re-acquire WL handle

USB.begin() → MSC callbacks read/write Flash via this handle

Why can't they share? FFat internally encapsulates FATFS + WL and exclusively owns the WL handle without exposing it. If we call `wl_mount()` before `FFat.end()`, it will fail (handle already in use). Therefore, FFat must be completely unmounted first, then WL mounted separately.

2.3 Wear-Leveling

SPI Flash has an erase/write cycle limit per sector (about 100,000 times). If a sector is repeatedly erased/written (e.g., PC repeatedly modifies the same file), that sector will fail quickly. The WL layer solves this through three mechanisms:

1. **Address Remapping:** Logical sector numbers don't directly map to physical sector numbers; WL maintains a mapping table
2. **Write Distribution:** Each write allocates a different physical sector, avoiding "hot spot" concentrated wear
3. **Garbage Collection:** Marks old pages as invalid, background async erase and recovery

Logical sectors (seen by PC)	Physical sectors (actual Flash)
LBA 0	→ Physical sector 12
LBA 1	→ Physical sector 47
LBA 2	→ Physical sector 3
...	(each write may remap)

2.4 NOR Flash Writes Must Erase First

NOR Flash physical constraint: write operations can only change bits from 1 to 0, not from 0 to 1. The only way to change 0 back to 1 is to **erase the entire sector** (typically 4KB or larger), after which all bits return to 1.

Current Flash byte value: 1011 0011 (existing data)

Value PC wants to write: 0100 1100

Direct write: 1011 0011 | 0100 1100 = 1111 1111 ← 1→0 succeeded, but 0→1 didn't work!

Correct flow: Erase entire sector first (all become 0xFF) → Write → 0100 1100 ✓

This is why `onwrite` callback must **first** `wl_erase_range()` **then** `wl_write()` — if you forget to erase and write directly, the write silently fails, manifesting as file corruption on the PC side.

2.5 Reason for ESP-IDF API Mixing

Arduino framework provides `FFat` for file-level operations (`File.open` / `File.write`), but USB MSC's `onRead` / `onwrite` callbacks need **block-level** (sector-level) Flash access — directly reading/writing 512 bytes given an LBA address. Arduino has no block-level Flash API, so three ESP-IDF interfaces must be mixed:

Function	Purpose	Arduino Equivalent
<code>esp_partition_find_first()</code>	Find FAT partition in partition table	None
<code>wl_mount()</code> / <code>wl_size()</code>	Mount wear-leveling layer, get available sector count	None
<code>wl_read()</code> / <code>wl_write()</code> / <code>wl_erase_range()</code>	Block-level Flash read/write/erase	None

This is not code style mixing but a limitation of ESP32 platform architecture — TinyUSB MSC must access Flash via block-level interface, and Arduino lacks this layer of abstraction.

3. Hardware Dependencies

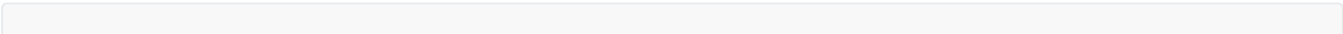
Pin	Peripheral	Configuration	Description
GPIO19	USB D-	USB OTG	Native USB port differential data negative
GPIO20	USB D+	USB OTG	Native USB port differential data positive

ESP32-S3 USB OTG Hardware Features:

Feature	Parameter
USB Controller	USB 2.0 OTG Full-Speed (12 Mbps)
Built-in PHY	Yes (no external USB transceiver needed)
USB Pins	GPIO19 (D-) / GPIO20 (D+), fixed, cannot remap
Supported Device Classes	CDC (serial), MSC (storage), HID (keyboard/mouse), DFU (firmware upgrade)
Power Mode	Bus-Powered, max 500mA
Low-level Protocol Stack	TinyUSB (built into ESP32 Arduino Core)

Must use the native USB port to connect to PC. On-board USB-UART port (GPIO43/44) is only for serial monitor. Also requires a `data`, `fat` type partition in `partitions.csv`.

4. Project Architecture and Flow



```

setup()
├─ writewelcomeFile()           // ① Write welcome.txt using FFat
│   ├─ FFat.begin(true, "/ffat", 5, "storage")
│   ├─ File.write(welcome.txt)
│   └─ FFat.end()               // Release WL handle
│
├─ esp_partition_find_first()    // ② Find FAT partition
├─ wl_mount(part, &wl_handle)    // ③ Remount WL (for MSC use)
│
├─ MSC configuration            // ④ Register callbacks + device info
│   ├─ MSC.onRead(onRead)
│   ├─ MSC.onWrite(onWrite)
│   ├─ MSC.vendorID/productID
│   └─ MSC.begin(sectorCount, 512)
│
└─ USB.begin()                  // ⑤ Start USB device stack → PC enumeration

[USB MSC callbacks - background]
├─ onRead → wl_read()
├─ onwrite → wl_erase_range() + wl_write()
└─ onStartStop → return true

```

5. Source Code

Source Code Location: `Source code → Arduino → 2.Serial communication → 5.USB → 5.USB.ino`

5.1 Write Preset File

Use Arduino FFat API to create `welcome.txt` in the partition, release WL handle after operation.

```

static void writewelcomeFile() {
    if (!FFat.begin(true, "/ffat", 5, "storage")) return;
    File f = FFat.open("/welcome.txt", FILE_WRITE);
    f.println("welcome to Yahboom USB Mass Storage Demo!");
    f.close();
    FFat.end(); // Unmount → Release WL handle, make way for MSC
}

```

`FFat.end()` is the critical step — FFat and USB MSC cannot simultaneously hold the WL handle.

5.2 MSC Read/Write Callbacks

PC calls `onRead` once per sector read, and `onwrite` once per sector write. Address calculation: `1ba * 512 + offset`.

```

static int32_t onRead(uint32_t lba, uint32_t offset, void *buffer, uint32_t bufsize) {
    return wl_read(wl_handle, (size_t)lba * 512 + offset, buffer, bufsize)
        == ESP_OK ? (int32_t)bufsize : -1;
}

static int32_t onWrite(uint32_t lba, uint32_t offset, uint8_t *buffer, uint32_t bufsize) {
    wl_erase_range(wl_handle, (size_t)lba * 512, 512); // NOR Flash must erase before write
    return wl_write(wl_handle, (size_t)lba * 512 + offset, buffer, bufsize)
        == ESP_OK ? (int32_t)bufsize : -1;
}

```

`wl_erase_range` erases the entire sector (512 bytes). Writing without erasing causes Flash controller to silently fail, manifesting as file corruption on PC side.

5.3 `setup()` — WL Mount and MSC Startup

Take over WL handle from FFat, configure MSC callbacks, start USB.

```

void setup() {
    Serial.begin(115200);

    writeWelcomeFile(); // ① Write preset file

    const esp_partition_t *part = esp_partition_find_first(
        ESP_PARTITION_TYPE_DATA, ESP_PARTITION_SUBTYPE_DATA_FAT, NULL);
    wl_mount(part, &wl_handle); // ② Remount WL

    uint32_t sectorCount = wl_size(wl_handle) / 512;

    MSC.vendorID("Espressif");
    MSC.productID("ESP32-S3 USB MSC");
    MSC.onRead(onRead);
    MSC.onWrite(onWrite);
    MSC.onStartStop(onStartStop);
    MSC.mediaPresent(true);
    MSC.iswritable(true);
    MSC.begin(sectorCount, 512); // ③ Start MSC

    USB.begin(); // ④ Start USB device stack
    Serial.println("[READY] Plug in the native USB cable");
}

```

6. Operation Flow

6.1 Compile and Flash

Arduino Flash Process: See 1. Before Development → 2. Build and Flash.

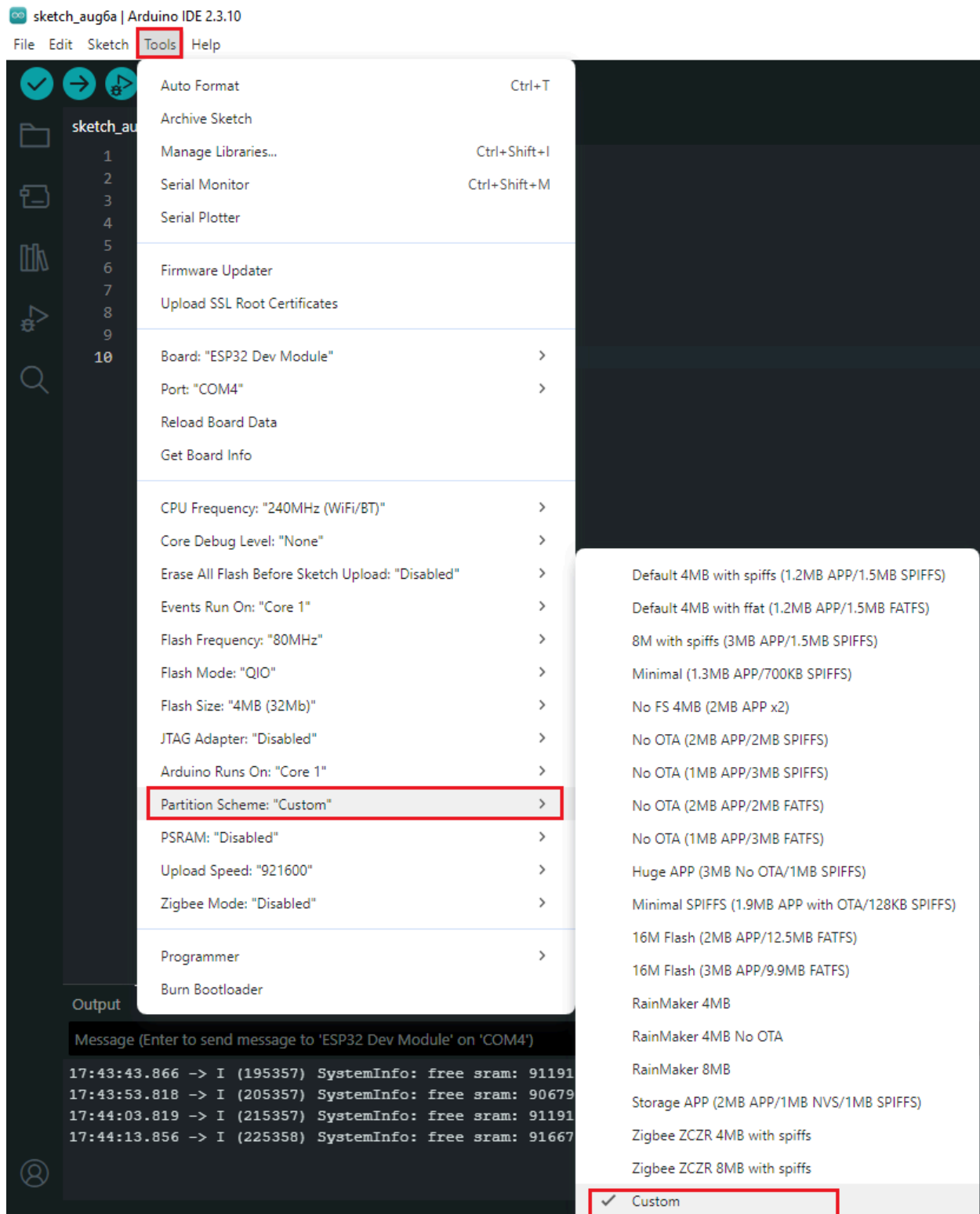
1. Confirm `partitions.csv` has `storage`, `data`, `fat` type partition
2. Open `5.USB.ino`

3. Before flashing, connect native USB port (GPIO19/20) to PC with Type-C data cable
4. After flashing and powering on, PC immediately recognizes the flash drive

Custom Partition Table Setup:

Arduino IDE's default partition scheme doesn't include FAT type partition, requiring manual custom partition table specification:

1. Tools → Partition Scheme → Select "Custom"



2. Create `partitions.csv` in the Arduino project directory with the following content:

#	Name,	Type,	SubType,	Offset,	Size
nvs,	data,	nvs,	0x9000,	0x6000,	
factory,	app,	factory,	0x10000,	1M,	
storage,	data,	fat,	,	1M,	

The partition table defines three partitions: NVS (Wi-Fi/calibration data storage), factory (firmware program), storage (FAT filesystem for MSC flash drive emulation). The storage partition type must be `data`, `fat`, size 1MB, offset left blank means immediately after the previous partition.

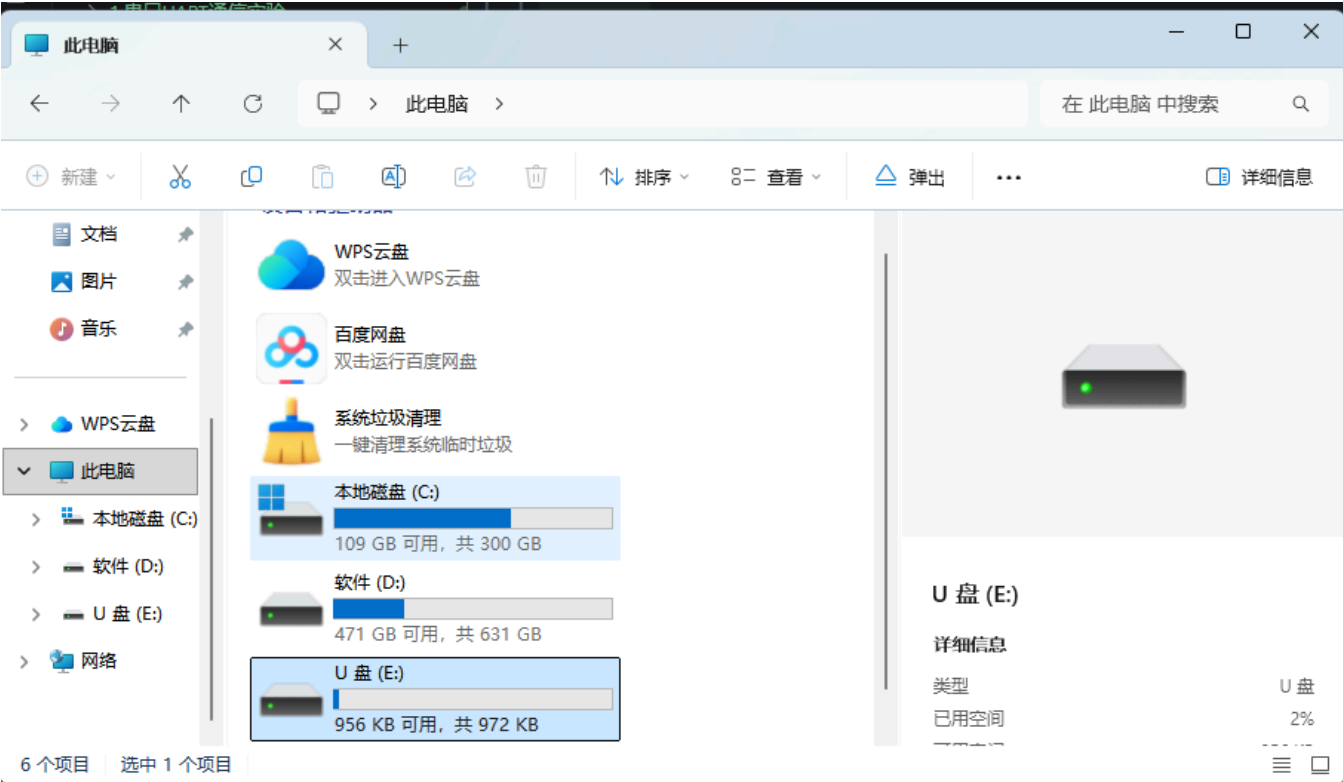
6.2 Normal Phenomena

Open **Serial Monitor** (115200 bps), power on, initialize and print logs:

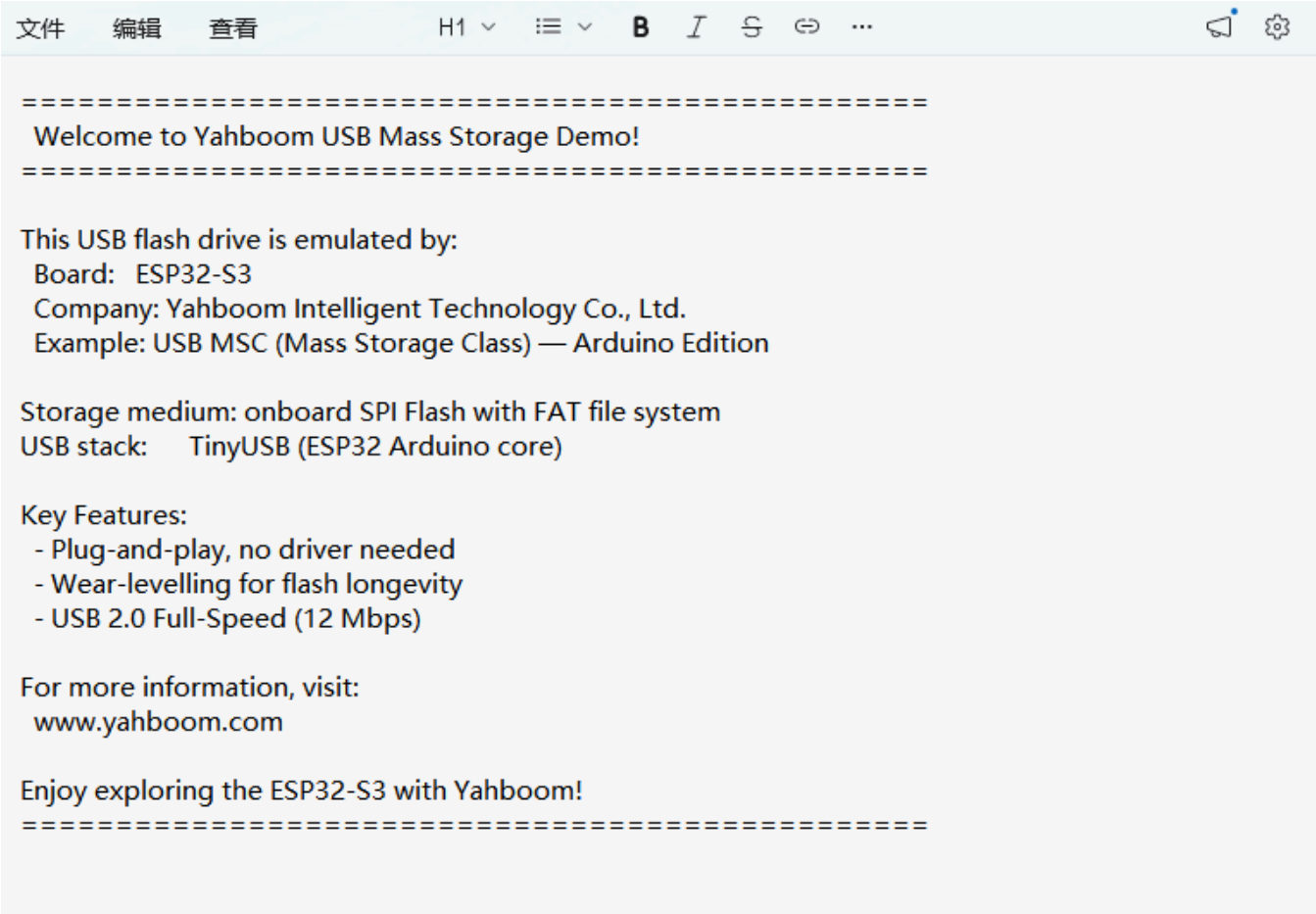
```
=====
ESP32-S3 USB MSC - Arduino Edition
=====
[FFat] Mounted, writing welcome.txt...
[FFat] Created welcome.txt
[FFat] Unmounted (will remount WL for USB MSC)
[WL] Found partition: storage (1048576 bytes)
[WL] Mounted, sectors: 250 (each 4096 bytes)
[USB] Started - plug in the native USB cable
[READY]
```

```
=====
ESP32-S3 USB MSC - Arduino Edition
=====
[FFat] Mounted, writing welcome.txt...
[FFat] Created welcome.txt
[FFat] Unmounted (will remount WL for USB MSC)
[WL] Found partition: storage (1048576 bytes)
[WL] Mounted, sectors: 250 (each 4096 bytes)
[USB] Started - plug in the native USB cable
[READY]
```

After power-on, PC immediately recognizes it as a FAT-formatted flash drive with a readable/writable `welcome.txt` file inside.



welcome.txt



6.3 Verification Methods

Operation	Expected Result
USB cable connects to PC	PC recognizes as "ESP32-S3 USB MSC" flash drive
Open welcome.txt	Shows Yahboom welcome content
Create new file on PC side	Writes normally, persists after power cycle
Unplug USB cable	Serial prints <code>[USB] UNPLUGGED</code>

7. Common Issues

Phenomenon	Cause	Solution
Plugging USB cable PC has no response	Using USB-UART port instead of native USB port	Switch to native USB port GPIO19/20
PC prompts "Disk needs formatting"	FAT partition not created or corrupted	Check <code>partitions.csv</code> + <code>FFat.begin(true, ...)</code>
PC writes file then restart file disappears	<code>onwrite</code> wrote without erasing	Confirm <code>wl_erase_range</code> is before <code>wl_write</code>
<code>wl_mount</code> fails	WL handle still held by FFat not released	Confirm <code>FFat.end()</code> is before <code>wl_mount</code>
Serial doesn't print <code>[USB] PLUGGED</code>	USB CDC and MSC share same USB port	Disable USB CDC On Boot, use <code>Serial0</code> instead